Stacks, Queues and Deque

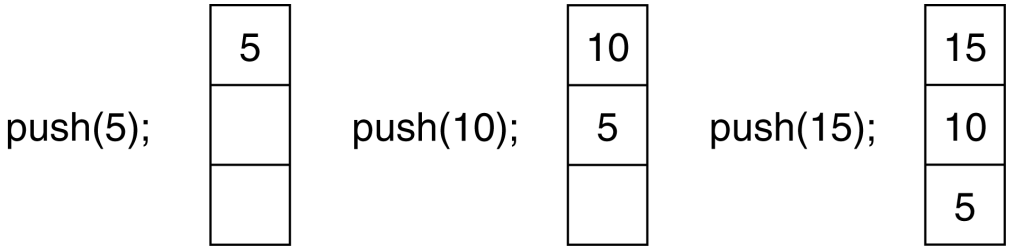
• Stack A stack is a data Structure in which insertion and deletion take place at the same end. Stacks are known as LIFO (Last In, First Out) lists. The last element inserted will be the first to be retrieved. We can only add/remove/examine the last element added (the "top").

• STACK OPERATIONS: ● Push: To insert an item from Top of stack is called push operation. ● POP: To put-off, get or remove some item from top of the stack is the pop operation. ● IsEmpty: Stack considered empty when there is no item on top. IsEmpty operation return true when there is no item in stack else false. ● IsFull: Stack considered full if no other element can be inserted on top of the stack.



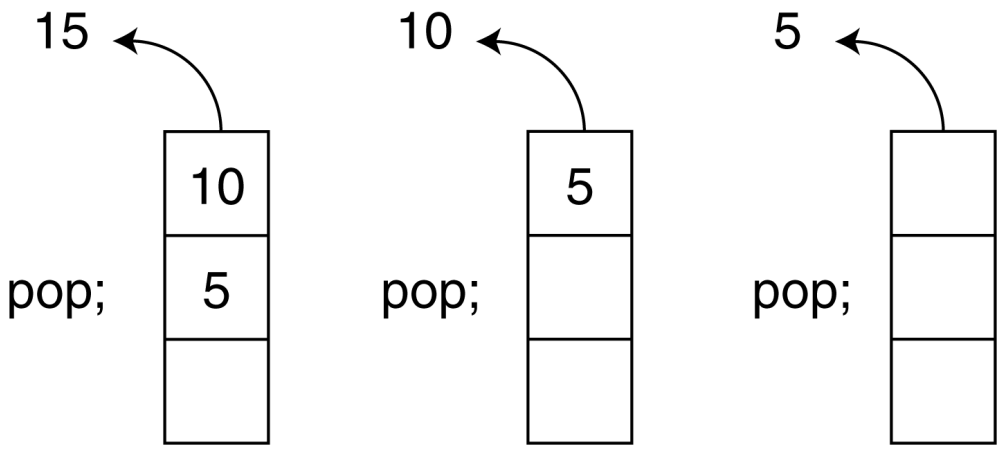
Example of Push and Pop

- Push Add an element to the top of the stack. Pop Remove the element at the top of the stack.
- top
- IsEmpty stack



push an element

- push another
- pop
- Example of Push and Pop The Push Operation Suppose we have an empty integer stack that is capable of holding a maximum of three values. With that stack we execute the following push operations. push(5); push(10); push(15);



The state of the stack after each of the push operations:

- The Pop Operation Now, suppose we execute three consecutive pop operations on the same stack:

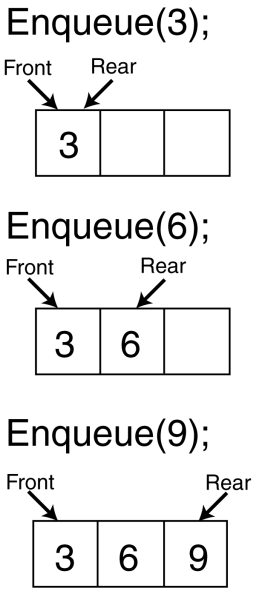
```
This is a sample program to demonstrate push and pop functionality in Stack in Java: public class StackDemo { private int size = 3; int arr[] = new int[size]; int front = -1; // variable declaration and initialize with -1 public void push(int pushedElement) { // push method if (front < size - 1) { // when front of stack is less size-1 front++; // increment the top arr[front] = pushedElement; // push value at array of front where front is index System.out.println("Element " + pushedElement + " is pushed to Stack !"); printElements(); // calling statement of print element method } else { System.out.println("Stack Overflow !"); // print when stack is full } }
```

```
Stack Underflow !
Element 23 is pushed to Stack !
Elements in stack :
23
Element 2 is pushed to Stack !
Elements in stack :
23
2
Element 73 is pushed to Stack !
Elements in stack :
23
2
73
Stack Overflow !
Pop operation done !
Pop operation done !
Pop operation done !
Stack Underflow !
```

public void pop() { // pop method if (front >= 0) { // if condition true when front is greater than 0 front--; // decrement the front System.out.println("Pop operation done !"); } else { System.out.println("Stack Underflow !"); } } public void printElements() { // display function heading if (front >= 0) { System.out.println("Elements in stack :"); for (int i = 0; i <= front; i++) { // loop start from 0 index to the front of the stack System.out.println(arr[i]); // printing statement } } }

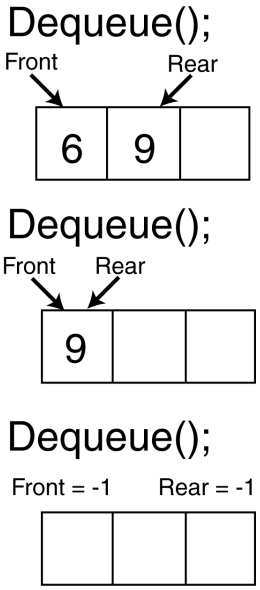


public static void main(String[] args) { StackDemo stackDemo = new StackDemo(); stackDemo.push(23); // calling statement of push method with parameter (passing value) stackDemo.push(2); stackDemo.push(73); stackDemo.push(21); stackDemo.pop(); // calling statement of pop method stackDemo.pop(); stackDemo.pop(); stackDemo.pop(); } }



Output If everything goes right you will see following output on console demonstrating all possible cases in Stack Implementation.

• Queue A queue is a Data structure, in which insertion is done at one end, while deletion is performed at the other end. ---Accessing the elements of queues follows a First In, First Out (FIFO) order. --Like customers standing in a line in a store, the first customer in is the first customer served. --We can only add to the end of the queue, and can only examine/remove the front of the queue.



Enqueue and Dequeue

• Queue operations: Enqueue and Dequeue Like lines in a store, a queue has a front and a rear. Enqueue – insert an element at the rear of the queue Dequeue – remove an element from the front of the queue

• Insert (Enqueue)

• Remove (Dequeue)

```
Underflow !
Element 23 is pushed to Queue !
Elements in Queue :
23
Element 2 is pushed to Queue !
Elements in Queue :
23
2
Element 73 is pushed to Queue !
Elements in Queue :
23
2
73
Overflow !
Pop operation done !
Elements in Queue :
2
73
Pop operation done !
Elements in Queue :
73
Pop operation done !
Underflow !
```

rear

- front
- Example of Enqueue and Dequeue: Suppose we have an empty static integer queue that is capable of holding a maximum of three values. With that queue we execute the following enqueue operations. Enqueue(3); Enqueue(6); Enqueue(9);
- Example of Enqueue and Dequeue: The state of the queue after each of the enqueue operations.

Example of Enqueue and Dequeue: Now the state of the queue after each of three consecutive dequeue operations

- Example Program for Stack implementation using Generics

```
import java.util.ArrayList; interface Container<T> { void push(T item); T pop(); boolean isEmpty(); } class Stack<T> implements Container<T> { private ArrayList<T> items = new ArrayList<>(); public void push(T item) { items.add(item); } public T pop() { if (items.isEmpty()) throw new RuntimeException("Stack empty"); return items.remove(items.size() - 1); }
```

public boolean isEmpty() { return items.isEmpty(); } // generic method inside a generic class public <U extends T> void pushAll(java.util.List<U> list) { for (U u : list) push(u); } } public class StackDemo { public static void main(String[] args) { Stack<Number> s = new Stack<>(); s.push(10); s.push(2.5); s.pushAll(java.util.List.of(1, 2, 3)); // U = Integer, a subtype of Number while (!s.isEmpty()) System.out.print(s.pop() + " "); } }

Queue Implementation of Array

• There algorithms to implement Enqueue and Dequeue When enqueuing, the front index is always fixed and the rear index moves forward in the array.

• Enqueue(3)

• Enqueue(6)

Enqueue(9)

- When dequeuing, the front index is fixed, and the element at the front the queue is removed. Move all the elements after it by one position.
- Dequeue()
- rear = -1

This is a sample program to demonstrate push and pop functionality in Queue in Java. public class QueueDemo { private int size = 3; // instance variable and size of of array int arr[] = new int[size]; // declaration of array and assigning size to array int front = -1; // front of queue int rear = 0; // rear of queue public void push(int pushedElement) { // insertion method declaration if (front < size - 1) { // front is less then size-1 front++; //increment the front arr[front] = pushedElement; // set element at array at front System.out.println("Element " + pushedElement + " is pushed to Queue !"); display(); // display method calling statement } else { System.out.println("Overflow !"); } }

public void pop() { // deletion method declaration if (front >= rear) { // true till front is greater or equal to rear rear++; System.out.println("Pop operation done !"); display(); // display method calling statement } else { System.out.println("Underflow !"); } } public void display() { // display method declaration if (front >= rear) { System.out.println("Elements in Queue : "); for (int i = rear; i <= front; i++) { // start from rear to front System.out.println(arr[i]); } } }

public static void main(String[] args) { QueueDemo queueDemo = new QueueDemo(); queueDemo.pop(); queueDemo.push(23); // calling statement of insertion method queueDemo.push(2); queueDemo.push(73); queueDemo.push(21); queueDemo.pop(); // calling statement of deletion method queueDemo.pop(); queueDemo.pop(); queueDemo.pop(); } }

- Output If everything goes right you will see following output on console demonstrating all possible cases in Queue Implementation

Deque

- A Deque (Double-Ended Queue) is a linear data structure that allows insertion and deletion of elements from both ends.
- Operations in Dequeue
- insertFront(x) → Insert an element at the front end. insertRear(x) → Insert an element at the rear end. deleteFront() → Delete an element from the front end. deleteRear() → Delete an element from the rear end. getFront() → Retrieve (but don't remove) the front element. getRear() → Retrieve (but don't remove) the rear element. isEmpty() → Check if the deque is empty. size() → Return the number of elements currently in the deque.

Stack, Queue and Deque in Java

- Structure
- Rule
- Insert

Remove

- Typical use
- Stack
- LIFO (Last In, First Out)

push (top)

- pop (top)
- Undo, recursion, expression evaluation
- Queue

FIFO (First In, First Out)

- enqueue (rear)
- dequeue (front)
- Scheduling, buffering, BFS

Double-ended: both ends allowed

- front and rear
- Sliding window, palindrome check, work stealing
- https://onlinegdb.com/uH3iVUpud Program with classes implemented for stack, queue and deque

Using the JDK built in classes

- ArrayDeque is the recommended choice for stack, queue and deque. https://onlinegdb.com/sZuQ8XqvLC
- Builtin Methods in ArrayDeque Class and Declaration
- Category

Method

- Syntax
- Returns
- If Deque Is Empty

addFirst

- dq.addFirst(E e)
- void
- n/a

offerFirst

- dq.offerFirst(E e)
- boolean
- addLast

dq.addLast(E e)

- offerLast
- dq.offerLast(E e)
- removeFirst

E x = dq.removeFirst()

- Throws NoSuchElementException
- pollFirst
- E x = dq.pollFirst()

Returns null

- removeLast
- E x = dq.removeLast()
- pollLast

E x = dq.pollLast()

- Examine
- peekFirst
- E x = dq.peekFirst()

peekLast

- E x = dq.peekLast()
- Stack (LIFO)
- push

dq.push(E e)

- E x = dq.pop()
- peek
- E x = dq.peek()

Deque<Integer> dq = new ArrayDeque<>(); // default capacity 16 Deque<String> dq2 = new ArrayDeque<>(50); // initial capacity hint

- Methods in ArrayDeque Class
- Queue (FIFO)
- offer

dq.offer(E e)

- add
- dq.add(E e)
- poll

E x = dq.poll()

- remove
- E x = dq.remove()
- Utility

size

- int n = dq.size()
- int
- Returns 0

isEmpty

- boolean b = dq.isEmpty()
- Returns true
- contains

boolean b = dq.contains(Object o)

- Returns false
- clear
- dq.clear()

No effect

- Traversal
- iterator
- Iterator<E> it = dq.iterator()

Iterator<E>

- hasNext() is false
- descendingIterator
- Iterator<E> it = dq.descendingIterator()

THE END